

Number Systems, Representation, and Arithmetic Hardware

Lecture Notes — Week 2

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 10, 2026

2.1 Introduction

Week 1 was about speed: it distinguished **architecture** (the interface) from **organization** (the implementation), established the performance equation, $\text{CPU time} = \text{IC} \times \text{CPI}/f$, and closed with **Amdahl's Law** — the unimproved fraction of a program sets a hard ceiling on the speedup any single improvement can deliver. The performance equation carries the assumption that we can count instructions and measure CPI. But a CPU cannot execute anything until the **data** it works on is represented in bits, and arithmetic is implemented in hardware. This chapter is about what the bits mean and how hardware computes with them. It answers six questions that form one thread: how we write numbers in binary, octal, and hex and convert between bases (Section 2.2); how hardware represents negative integers in two's complement (Section 2.3); when arithmetic fails (overflow) and how fractions are represented in fixed point (Section 2.4); how we build a circuit that actually adds (Section 2.5); how hardware multiplies signed numbers without a separate sign step, via Booth's algorithm (Section 2.6); and how hardware divides, by restoring and non-restoring division (Section 2.7).

Throughout the chapter we thread a single worked example: adding $+7 + (-3)$ in binary, bit by bit, then building the circuit that does it, and checking the result against the range of the format. The thread is closed and extended in Section 2.8, first when the same addition is run through a ripple-carry adder in hardware, and then when the same adder — reused with different control logic — is shown to be all that multiplication and division need.

2.2 Number Systems

2.2.1 Motivation: One Value, Many Alphabets

The decimal number 13 can be written with two symbols, or eight, or sixteen — each is the same value in a different alphabet. Decimal $13 = 1 \times 10^1 + 3 \times 10^0$. Binary $1101_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 13$. Octal 15_8 and hex D_{16} are each a shorthand for groups of bits. Hardware stores *bits*; the base is just how we write them down.

Definition (Positional Number System). *In base r , a number with digits $d_{n-1} \dots d_1 d_0$ has value*

$$d_{n-1}r^{n-1} + \dots + d_1r^1 + d_0r^0.$$

The base is written as a subscript: 1101_2 , 15_8 , D_{16} .

The common bases are $r = 2$ binary (digits 0,1), $r = 8$ octal (0–7), $r = 10$ decimal, and $r = 16$ hex (0–9, A–F). **Conversion** works by repeated division: dividing by r repeatedly yields the digits

from least to most significant. Hex and octal are just binary in **groups**: one hex digit is four bits, one octal digit is three bits.

Example 2.2.1 (Decimal to binary, and back). *Convert 53_{10} to binary by repeated division by 2: $53/2 = 26 \text{ r } 1$; $26/2 = 13 \text{ r } 0$; $13/2 = 6 \text{ r } 1$; $6/2 = 3 \text{ r } 0$; $3/2 = 1 \text{ r } 1$; $1/2 = 0 \text{ r } 1$. Reading the remainders bottom-up gives 110101_2 . Check by expanding: $1 \times 32 + 1 \times 16 + 0 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 = 53$. Binary $\leftrightarrow$ hex is trivial by grouping: $11010101_2 = D5_{16}$.*

2.2.2 How Many Bits Does It Take?

How many bits do you need to represent n distinct values? The answer is $\lceil \log_2 n \rceil$ bits. With k bits, the largest unsigned integer is $2^k - 1$: 8 bits give 0 to 255, 16 bits give 0 to 65535. This ceiling matters the moment we talk about **overflow**.

2.3 Signed Representation

2.3.1 Three Candidate Schemes for Negatives

Given n bits, how do we encode a sign? Three schemes compete. **Sign-magnitude** uses one sign bit plus $n - 1$ magnitude bits — simple, but it has two zeros (+0 and -0) and awkward addition. **One's complement** negates by flipping all bits — still two zeros, and addition needs an end-around carry. **Two's complement** negates by flipping all bits and adding 1 — it has one zero, and *addition just works*: $-x$ is defined so that $x + (-x) = 0$ in the same bit width, with the carry-out simply discarded. The ALU needs only an adder.

Definition (Two's Complement). *The two's complement of an n -bit number x is $-x \equiv 2^n - x \pmod{2^n}$, equivalently “flip all bits and add 1.” The representable range is $-2^{n-1} \leq x \leq 2^{n-1} - 1$; the sign bit is the MSB (0 for non-negative, 1 for negative). Adding a number to its two's complement gives all zeros — the “self-cancelling” property that makes subtraction free (P&H, Sec. 3.2, p.188; Hamacher Ch. 2) [3, 1].*

Example 2.3.1 (+7 and -3 in 8-bit two's complement). $+7 = 00000111_2$. To form -3: flip $00000111 \rightarrow 11111000$, add 1, giving 11111001_2 . Check by adding: $00000111 + 11111001 = 100000100$; discard the carry-out (the 9th bit) to get $00000100 = +4$, i.e. $7 + (-3) = 4$. Negative numbers are ordinary bits, and addition needs no special subtraction hardware — just discard the carry-out.

2.3.2 Negation as an Involution

-5 in 8-bit two's complement is 11111011 . Its own negation: flip to 00000100 , add 1, giving $00000101 = +5$. Negating twice returns the original — two's complement is an **involution**: $-(-x) = x$. This self-inverse property is why negation is easy to implement and verify.

2.4 Overflow and Fixed-Point

2.4.1 What Is Overflow?

Overflow occurs when the true result of an arithmetic operation cannot be represented in the format's range. For unsigned numbers, a carry out of the MSB is overflow. For two's complement, overflow is *not* the same as carry-out: a carry out of the MSB does not always mean overflow (recall

7 + (−3) above, which carried out yet was correct). The correct condition is that overflow occurs when the sign of the result is *wrong* — adding two positives gives a negative, or two negatives give a positive (P&H, Sec. 3.2, p.188) [3].

Example 2.4.1 (Overflow vs. carry in 8-bit two’s complement). *The 8-bit two’s complement range is −128 to +127.*

- +100 + +50: 0110 0100 + 0011 0010 = 1001 0110, which reads as −106 — *wrong sign, so overflow.*
- +127 + 1: 0111 1111 + 1 = 1000 0000 = −128 — *again wrong sign, overflow.*
- −1 + (−1): 1111 1111 + 1111 1111 = 1 1111 1110; *discard the carry to get 1111 1110 = −2 — correct.*

The rule of thumb: overflow happens exactly when the two operands share a sign bit and the result’s sign bit differs. Carry-out alone is not the test.

2.4.2 Fixed-Point Representation

A **fixed-point** number has an agreed position for the “binary point”:

$$b_{k-1} \dots b_0 . b_{-1} b_{-2} \dots b_{-m}, \quad \text{value} = \sum_i b_i 2^i.$$

For example, Q8.8 means 8 integer bits and 8 fraction bits, where each fractional bit is $2^{-1}, 2^{-2}, \dots$. Thus $0.5 = 0.1_2$, $0.25 = 0.01_2$, $0.75 = 0.11_2$. Fixed point is simple and fast — add/sub work on the same integer hardware — but the fixed point limits range and precision, which is why Week 3 moves to floating point (Hamacher Ch. 2; Mano Ch. 1) [1, 2].

Example 2.4.2 (A fixed-point addition). *Add $3.5 + 0.75$ in Q8.8. $3.5 = 0000\,0011.1000\,0000$; $0.75 = 0000\,0000.1100\,0000$. Adding bitwise: $0000\,0100.0100\,0000$, which reads as 4.25. The integer ALU adds fixed-point values unchanged — the programmer just agrees where the point is.*

2.4.3 Why Fixed Point Fails on Repeating Fractions

0.1 in binary is a **repeating** fraction ($0.000110011\dots_2$) — it does not terminate. Fixed point must **truncate**, so 0.1 is stored approximately, and repeated arithmetic accumulates error. This is the fundamental limitation that floating point (Week 3) manages but does not eliminate.

2.5 Adder Hardware

2.5.1 Motivation: From Addition to a Circuit

A human adds two numbers one column at a time, carrying. The smallest circuit that adds two *single bits* plus a carry-in is the **full adder**; chaining n of them builds a multi-bit adder. The **half adder** (HA) has two inputs A, B and outputs sum S and carry C ; the **full adder** (FA) has three inputs A, B, C_{in} and outputs sum S and carry C_{out} .

Definition (Half Adder and Full Adder).

$$HA: S = A \oplus B, \quad C = A \cdot B; \quad FA: S = A \oplus B \oplus C_{in}, \quad C_{out} = A \cdot B + C_{in} \cdot (A \oplus B).$$

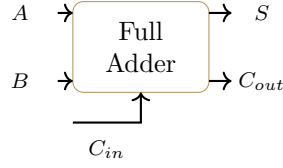


Figure 2.1: The full adder as a block: A, B and C_{in} in, S and C_{out} out.

The full adder can be drawn as a single block (Figure 2.1): inputs A and B enter from the left, C_{in} enters from below, and the outputs S and C_{out} exit to the right.

The full adder's truth table (Table 2.1) has the pattern: count the 1s among (A, B, C_{in}) — if odd, $S = 1$; if two or more, $C_{out} = 1$.

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 2.1: Full adder truth table.

2.5.2 The Ripple-Carry Adder

The **ripple-carry adder** chains n full adders so that each FA's C_{out} feeds the next FA's C_{in} (Figure 2.2). The carry **ripples** from the least-significant to the most-significant bit — hence the name — and the final C_{out} is the unsigned overflow flag.

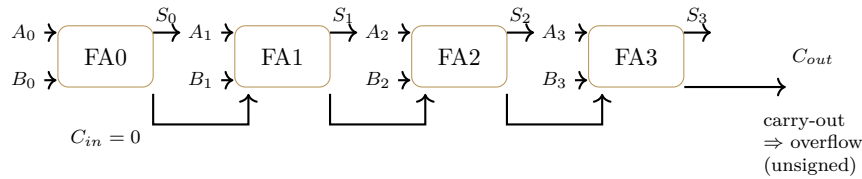


Figure 2.2: The ripple-carry adder: full adders chained by their carry signals.

Example 2.5.1 (Adding $7 + (-3)$ in hardware). $+7 = 0111$, $-3 = 1101$ (*4-bit two's complement*). Add with a 4-bit ripple-carry adder, $C_{in} = 0$: bit 0: $1 + 1 = 0$ carry 1; bit 1: $1 + 0 + 1 = 0$ carry 1; bit 2: $1 + 1 + 1 = 1$ carry 1; bit 3: $0 + 1 + 1 = 0$ carry 1. Result 0100 = +4, carry-out = 1 (discarded). One adder, same circuit, handles both signed and unsigned — the interpretation is the programmer's choice.

2.5.3 Subtraction Is Free

The ALU has only an adder. To compute $A - B$, one extra step suffices: subtract B 's two's complement, $A - B = A + (-B)$. Since $-B$ is “flip B + add 1,” this is implementable with a MUX

(choose B or $\bar{B}$) plus a forced carry-in of 1. Subtraction is therefore **free**: one adder plus a MUX, no separate subtractor needed (P&H, Sec. 3.2, p.188) [3].

2.6 Multiplication: Booth's Algorithm

2.6.1 Motivation: From Repeated Addition to a Faster Circuit

Multiplying $M \times Q$ by hand, the schoolbook way, adds M to itself once per *set* bit of Q , shifting each time. That shift-and-add approach always costs n additions for an n -bit Q , whether Q is $1111\dots$ or $0000\dots 1$, and it says nothing about *signed* operands — shift-and-add treats the sign bit like any other bit, which is wrong for two's complement. Section 2.5 already built signed addition and subtraction; the question is whether the *same* adder can drive multiplication too, without a special sign-handling step. **Booth's algorithm** answers yes: it recodes the multiplier so that runs of 1s become one subtract and one add instead of many adds.

Definition (Booth's Recoding Rule). *Let A be the accumulator (initialized to 0), Q the multiplier, Q_{-1} one extra bit (initialized to 0) that holds the previously shifted-out Q_0 , and M the multiplicand — all n bits wide. Repeat n times, examining the pair (Q_0, Q_{-1}) each cycle:*

- $(0, 0)$ or $(1, 1)$: no arithmetic — just arithmetic-shift-right (A, Q, Q_{-1}) by 1 bit.
- $(0, 1)$: $A \leftarrow A + M$, then arithmetic-shift-right.
- $(1, 0)$: $A \leftarrow A - M$, then arithmetic-shift-right.

After n cycles, the $2n$ -bit signed product is (A, Q) (P&H, Sec. 3.3, p.191; Hamacher Ch. 6, Sec. 6.4.1, p.380) [3, 1].

Example 2.6.1 ($13 \times (-6)$ by Booth's algorithm). *Use 5-bit two's complement: $M = 13 = 01101$, $Q = -6 = 11010$. The cycle-by-cycle trace, examining (Q_0, Q_{-1}) and recording A after the shift, is:*

Cycle	(Q_0, Q_{-1})	Operation	A after shift
1	(0, 0)	shift only	00000
2	(1, 0)	$A \leftarrow A - M$, shift	11001
3	(0, 1)	$A \leftarrow A + M$, shift	00011
4	(1, 0)	$A \leftarrow A - M$, shift	11011
5	(1, 1)	shift only	11101

Reading off $(A, Q) = 1110110010_2 = -78$, so $13 \times (-6) = -78$, which checks out.

The hardware for this algorithm is compact because it reuses the adder from Section 2.5 (Figure 2.3): each cycle, an Add/Sub-with- M block updates A by $\pm M$ or leaves it unchanged based on (Q_0, Q_{-1}) , and then the whole (A, Q, Q_{-1}) register shifts arithmetically right by one bit.

2.6.2 When Booth's Saves Nothing

Booth's algorithm always runs n cycles, the same as plain shift-and-add, so where is the saving? The saving is in **arithmetic operations**, not cycles: a run of consecutive 1s in Q becomes one subtract (at the run's start) and one add (just past its end) instead of one add per 1-bit. The worst case is alternating bits ($0101\dots$), where every cycle sees $(0, 1)$ or $(1, 0)$, so Booth's does an add/sub every cycle — the same as shift-and-add. Booth's algorithm is therefore never worse, and strictly better whenever Q has a run of two or more consecutive equal bits.

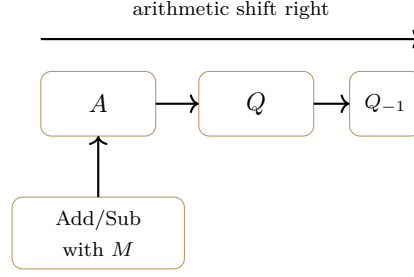


Figure 2.3: Booth’s algorithm datapath: the $A/Q/Q_{-1}$ shift register with an Add/Sub-with- M block feeding A — cf. P&H, Sec. 3.3, p.191 [3].

2.7 Division Algorithms

2.7.1 Motivation: Division Is Subtraction, Repeated

Long division by hand brings down a digit, subtracts the divisor as many times as it fits, records the count, and repeats. In binary, “as many times as it fits” collapses to *once or zero times per bit*, so division becomes shift, subtract, and a one-bit decision. Two hardware strategies differ only in what happens when the subtraction goes negative: **restoring division** subtracts, and if the result is negative, adds the divisor back (“restores”) before continuing; **non-restoring division** never restores — it carries the negative partial remainder forward and flips the next operation instead.

2.7.2 Restoring Division

Definition (Restoring Division). *Registers: A (remainder, initialized to 0), Q (dividend, becomes the quotient), M (divisor). Repeat n times:*

1. *Shift (A, Q) left by 1 bit.*
2. *$A \leftarrow A - M$.*
3. *If $A < 0$: **restore** $A \leftarrow A + M$ and set $Q_0 = 0$. Else: set $Q_0 = 1$.*

After n cycles, Q holds the quotient and A the remainder. Every cycle does a subtract, and on average half the time an extra restoring add — up to two arithmetic operations per cycle (P&H, Sec. 3.4, p.197; Hamacher Ch. 6, Sec. 6.6, p.390-392) [3, 1].

2.7.3 Non-Restoring Division

Definition (Non-Restoring Division). *Same registers as restoring division. Repeat n times:*

1. *Shift (A, Q) left by 1 bit.*
2. *If $A \geq 0$: $A \leftarrow A - M$. Else: $A \leftarrow A + M$.*
3. *If $A \geq 0$: set $Q_0 = 1$. Else: set $Q_0 = 0$.*

After n cycles, if $A < 0$ a single final correction $A \leftarrow A + M$ recovers the true remainder; Q is already the correct quotient. Exactly one arithmetic operation happens per cycle, always — no restore step.

The two strategies are compared side by side in Figure 2.4: both repeat n times, but restoring division can spend up to two arithmetic operations per cycle while non-restoring always spends exactly one (plus at most one final correction).

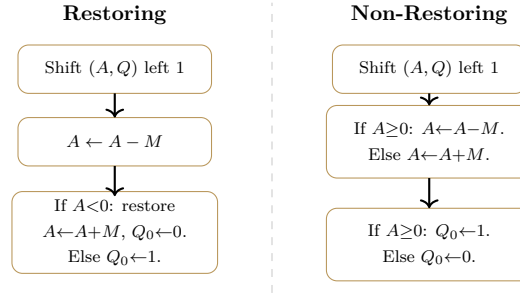


Figure 2.4: Restoring vs. non-restoring division, side by side. Both repeat n times; restoring can spend 2 arithmetic ops/cycle, non-restoring always spends exactly 1 (plus one final correction) — cf. P&H, Sec. 3.4, p.197 [3].

Example 2.7.1 ($13 \div 3$, both algorithms). Use 4-bit unsigned arithmetic: $Q = 13 = 1101$ (dividend), $M = 3 = 0011$ (divisor). Both algorithms run 4 cycles and must agree on the answer: quotient $Q = 4 = 0100$, remainder $A = 1 = 0001$. Check: $13 = 4 \times 3 + 1$. Both algorithms converge on quotient 4, remainder 1; non-restoring just gets there with fewer arithmetic operations.

The cycle-by-cycle trace for **restoring division**, following the registers A , Q , M and the three-step procedure above, is:

Cycle	Shift (A, Q)	$A \leftarrow A - M$	Decision	A, Q after
1	$A=0001, Q=1010$	$A = 1110$ (neg.)	restore, $Q_0=0$	$A=0001, Q=1010$
2	$A=0011, Q=0100$	$A = 0000$ (≥ 0)	keep, $Q_0=1$	$A=0000, Q=0101$
3	$A=0000, Q=1010$	$A = 1101$ (neg.)	restore, $Q_0=0$	$A=0000, Q=1010$
4	$A=0001, Q=0100$	$A = 1110$ (neg.)	restore, $Q_0=0$	$A=0001, Q=0100$

After 4 cycles, $Q = 0100 = 4$ (quotient) and $A = 0001 = 1$ (remainder), with no final correction needed.

The cycle-by-cycle trace for **non-restoring division**, using the conditional add/subtract and no restore step, is:

Cycle	Shift (A, Q)	Operation	Decision	A, Q after
1	$A=0001, Q=1010$	$A \geq 0: A \leftarrow A - M = 1110$	$A < 0 \Rightarrow Q_0=0$	$A=1110, Q=1010$
2	$A=1101, Q=0100$	$A < 0: A \leftarrow A + M = 0000$	$A \geq 0 \Rightarrow Q_0=1$	$A=0000, Q=0101$
3	$A=0000, Q=1010$	$A \geq 0: A \leftarrow A - M = 1101$	$A < 0 \Rightarrow Q_0=0$	$A=1101, Q=1010$
4	$A=1011, Q=0100$	$A < 0: A \leftarrow A + M = 1110$	$A < 0 \Rightarrow Q_0=0$	$A=1110, Q=0100$

After 4 cycles $A = 1110$ is negative, so a final correction applies: $A \leftarrow A + M = 1110 + 0011 = 0001$. The quotient $Q = 0100 = 4$ was already correct; the corrected remainder is $A = 0001 = 1$ — the same answer as restoring division, reached with exactly one arithmetic operation per cycle plus one final correction, instead of restoring division's occasional second operation.

2.7.4 Efficiency, Not Correctness, Is the Difference

Restoring division does a subtract every cycle, plus a restoring add whenever $A < 0$: up to 2 operations per cycle. Non-restoring division does exactly one add-or-subtract every cycle, plus

at most one final correction — never more than restoring, and often fewer. Both produce the same quotient and remainder either way, so non-restoring is a pure efficiency win, not a different algorithm.

2.8 Synthesis and Summary

2.8.1 The Thread, Closed and Extended

The chapter’s running thread — adding $+7 + (-3)$ — used every tool from the first four sections. We represented both operands in two’s complement (0111 and 1101), recognized that addition needs no special subtraction hardware, ran the bits through a ripple-carry adder, discarded the carry-out, and checked the sign to confirm no overflow. The result, 0100 = +4, is the same number a human would compute — but now it is the output of a specific circuit.

The same adder, driven by different control logic, extends the thread to multiplication and division. Booth’s algorithm examines (Q_0, Q_{-1}) each cycle — add, subtract, or no-op — then arithmetic-shift-rights the result, handling signed operands with the same adder and no special sign step: it computes $13 \times (-6) = -78$. Division reduces to shift, subtract, and a one-bit decision, differing between restoring (subtract, test, restore-if-negative) and non-restoring (conditional add/subtract, no restore) only in efficiency, not in the answer: both compute $13 \div 3 = 4$ remainder 1. No new arithmetic hardware is needed beyond the adder built in Section 2.5.

We now have all four integer operations — add, subtract, multiply, divide — in hardware. But fixed point handles fractions only with fixed range and precision, and real numbers span enormous ranges (from 10^{-300} to 10^{300} in science) that one fixed point cannot cover. Week 3 builds **floating point**: the IEEE 754 representation — sign, exponent, mantissa — trading a little precision for a huge range, plus the normalization and the arithmetic.

2.8.2 Summary

Number systems: $\text{value} = \sum d_i r^i$; convert by repeated division; hex/octal group bits. **Two’s complement:** $-2^{n-1} \leq x \leq 2^{n-1} - 1$; negate by flip + 1; addition needs no special subtraction hardware. **Overflow:** carry-out $\neq$ overflow; a wrong result sign is the test. **Fixed point:** an agreed binary point; simple but limited range and precision. **Adder:** HA/FA equations; the ripple-carry chain; $A - B = A + (-B)$. **Booth’s algorithm:** (Q_0, Q_{-1}) recoding — (0, 1) adds, (1, 0) subtracts, (0, 0)/(1, 1) shift only; never worse than shift-and-add. **Division:** restoring subtracts then conditionally restores (up to 2 ops/cycle); non-restoring always does exactly 1 op/cycle.

References

- [1] V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky. *Computer Organization*. McGraw-Hill, 5th edition, 2002.
- [2] M. Morris Mano. *Computer System Architecture*. Prentice-Hall, 3rd edition, 1993.
- [3] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, arm edition (5th) edition, 2017.